University of Essex

School of Computer Science and Electronic Engineering

CE315-6-AU: Mobile Robotics

Assignment: Simulated Robot Navigation

Registration Number: 2316794

Date: Autumn 2025

Contents

1 Introduction

The domain of mobile robotics, an interdisciplinary pursuit, integrates principles from mechanical engineering, computer science, and control theory to engineer autonomous entities endowed with the capacity for perception, data processing, and action within physical extents. This document delineates the conceptualization, realization, and assessment of a simulated robotic navigation framework, leveraging the Robot Operating System (ROS) and the Gazebo simulation environment. The principal aim involves the programmatic development of a mobile robot to achieve autonomous transit from an initial locus, traversing a sequence of designated waypoints, to a terminal charging facility, by employing sensor-derived information for spatial positioning and environmental mapping.

To furnish a foundational understanding for the implementation methodologies elaborated upon in subsequent sections, this introductory segment offers a comprehensive exposition and comparative analysis of the foundational constituents of mobile robotics: vehicular platforms, sensory apparatus, navigational paradigms, and pathfinding algorithms.

1.1 Mobile Robot Platforms and Mobility

The morphological design of a robotic system, commonly referred to as its platform, fundamentally influences its capacity for locomotion, its equilibrium, and the necessary control mechanisms. Robotic platforms are generally classified into two primary typologies: wheeled mechanisms and legged mechanisms, with each category exhibiting unique strengths and inherent constraints.

1.1.1 Wheeled Mobile Robots

Due to their straightforward mechanics, efficient energy usage, and simple operation on level terrain, wheeled robots are widely adopted in industrial and indoor environments.

- **Differential Drive:** This configuration utilizes two independently driven wheels typically placed on a common axis, supported by one or more castor wheels for stability. By varying the velocity of each wheel, the robot can move in straight lines, arcs, or rotate in place (zero turning radius). This high maneuverability makes it ideal for indoor environments, although it relies heavily on precise wheel synchronization to maintain a straight trajectory.

- **Car-like (Ackermann Steering):** Mimicking standard automobiles, these robots use separate mechanisms for driving and steering. While they offer superior stability at higher speeds and consistent four-wheel contact, they are non-holonomic systems with a minimum turning radius. This limits their ability to navigate tight corners or rotate in place compared to differential drive robots.

- **Tricycle Drive:** This setup involves a single driven and steered wheel (front or rear) with two passive wheels. It is mechanically simpler than car-like steering but can suffer from stability issues during sharp turns if the center of gravity is not properly managed.

1.1.2 Legged Robots

Legged robotic systems are engineered to navigate challenging, irregular, or discontinuous surfaces where wheeled mechanisms would be inadequate. These systems emulate biological movement patterns; however, they necessitate considerably more sophisticated control architectures.

- **Static Stability:** Robotic systems characterized by a quadrupedal or higher limb configuration (for instance, hexapodal robots) possess the capability to traverse environments while upholding a state of static stability. This is accomplished by guaranteeing that the center of mass of the robotic entity consistently lies within the confines of the support polygon established by the feet that are in contact with the substrate. This methodology facilitates the simplification of control mechanisms; however, it typically imposes restrictions on the velocity of movement.
- **Dynamic Stability:** Bipedal (two-legged) or hopping robotic systems are required to engage in active balance mechanisms to avert the risk of toppling, analogous to human beings. These systems exhibit dynamic stability, signifying that they can sustain an upright position exclusively during locomotion or through the continual modification of their posture. This characteristic facilitates enhanced agility and velocity; however, it necessitates rapid computational capabilities and sophisticated control algorithms.

1.2 Internal and External Sensors

The utilization of sensors is imperative for a robotic system to acquire the requisite information for the assessment of both its internal condition and the external environment. These sensors are categorized into two distinct types: internal sensors, also referred to as proprioceptive sensors, and external sensors, commonly known as exteroceptive sensors.

1.2.1 Internal Sensors

Internal sensors quantitatively assess parameters intrinsic to the robotic system, predominantly for the purpose of overseeing kinematics and operational condition.

- **Wheel Encoders:** Rotary encoders, electromechanical apparatuses affixed to wheel motors, serve to quantify rotational movement. The cumulative integration of these rotations over a temporal period enables the robot to ascertain its spatial location relative to an initial reference point, a process known as odometry. Nevertheless, the accuracy of encoder readings can be compromised by systematic errors arising from phenomena such as wheel slippage, substrate irregularities, or elastic deformation of pneumatic tires.
- **Inertial Measurement Units (IMUs):** Inertial Measurement Units (IMUs) commonly integrate accelerometers and gyroscopes to quantify linear acceleration and angular velocity. These devices are indispensable for discerning shifts in orientation and play a vital role in mitigating odometry errors, especially concerning the ascertainment of a robot's directional orientation.

1.2.2 External Sensors

The robot perceives its surroundings through external sensory apparatus, which is indispensable for both navigating around impediments and constructing spatial representations.

- **Sonar (Ultrasonic Sensors):** These sensors emit sound waves and measure the time-of-flight of the echo to calculate diThese transducers generate acoustic waves and ascertain the distance by measuring the interval between emission and reception of the reflected echo. Although economically viable, they exhibit limitations in angular precision and are susceptible to specular reflections. This phenomenon occurs when sound waves impinge upon smooth surfaces and are reflected at angles that preclude their return to the sensor, thereby rendering certain obstacles undetectable.
- **Lidar (Light Detection and Ranging):** Lidar technology employs laser pulses to ascertain distances with exceptional accuracy and rapidity. A two-dimensional Lidar system is designed to rotate, thereby generating a planar representation of its surroundings. Its accuracy and operational range render it the established standard for environmental mapping and autonomous navigation, notwithstanding its typically higher cost compared to sonar systems.
- **Cameras (Vision Sensors):** Cameras are capable of capturing extensive visual information, encompassing aspects such as color and texture. Utilizing computer vision methodologies (like stereo vision), they can assess depth and recognize objects. Although cameras yield the most comprehensive data, they necessitate considerable computational resources to analyze images for navigation objectives.

1.3 Navigation Approaches

Navigation enables a robot to ascertain its location (localization) and devise a path towards a target. The methods employed differ according to the information accessible regarding the surroundings.

- **Odometry (Dead Reckoning):** This approach depends exclusively on internal sensors (encoders/IMU) for position calculation. Although it is straightforward to implement and does not necessitate an external reference, it is plagued by unbounded error accumulation. A minor angular error at the outset can result in a significant position error over extended distances, rendering it inadequate as a standalone solution for long-term navigation.
- **Map-Based Navigation:** The robot employs a previously established map of its surroundings. By juxtaposing real-time sensor data (such as Lidar scans) with the familiar map, the robot is able to rectify its odometry inaccuracies and determine its position with precision. Although this approach is reliable, it becomes ineffective if there are substantial alterations in the environment or if a map is not accessible.
- **SLAM (Simultaneous Localization and Mapping):** In unfamiliar settings, a robot is required to create a map while concurrently identifying its position within that map. SLAM algorithms employ probabilistic techniques (such as Particle Filters or Kalman Filters) to manage the uncertainty associated with both the robot's

movement and sensor readings. Although this method is the most adaptable, it demands significant computational resources.

1.4 Path Planning Methods

Path planning involves calculating a route that avoids collisions, starting from a designated point and leading to a specified destination.

- **Reactive Methods (e.g., Bug Algorithms):** These algorithms operate without the need for a map. The robot advances straight towards the target until it meets an obstacle, at which point it navigates around it (for instance, by adhering to the wall) until the route to the target is unobstructed once more. They are low in computational cost and effectively manage dynamic obstacles; however, they frequently yield inefficient and suboptimal paths.
- **Global/Graph Search Methods:** These require a known map.
  - **Dijkstra's Algorithm:** This algorithm investigates the map in every direction from the initial node to identify the shortest route to the target. While it ensures an optimal solution, it proves to be inefficient in extensive areas due to its indiscriminate expansion.
  - **A* (A-Star) Algorithm:** A* enhances Dijkstra's algorithm by incorporating a heuristic (for instance, the direct distance to the target) to direct the search process. This method prioritizes the exploration of routes that are more likely to reach the destination, thereby identifying the optimal path considerably quicker than Dijkstra's algorithm.
  - **Potential Fields:** This method conceptualizes the robot as a particle navigating through a force field; the target exerts an attractive force on the robot, whereas obstacles generate a repulsive force. Although this approach is mathematically sophisticated and beneficial for immediate obstacle avoidance, it may encounter issues with 'local minima,' causing the robot to become ensnared in a state of equilibrium within the force field prior to achieving its objective.

2 Implementation of Odometry Control Strategy in Lab 3

Lab 3 focused on implementing a dead reckoning navigation strategy using strictly odometry data. By monitoring coordinates (x, y) and heading (ψ) via wheel encoders, the robot followed a prescribed path using a "Turn-then-Move" method to decouple rotational and translational movements.

2.1 Control Logic Flowchart

The control architecture employs a Finite State Machine (FSM) operating at a 10Hz sampling frequency. The robot transitions through distinct motion "Stages" as sensor data satisfies predefined coordinate thresholds.

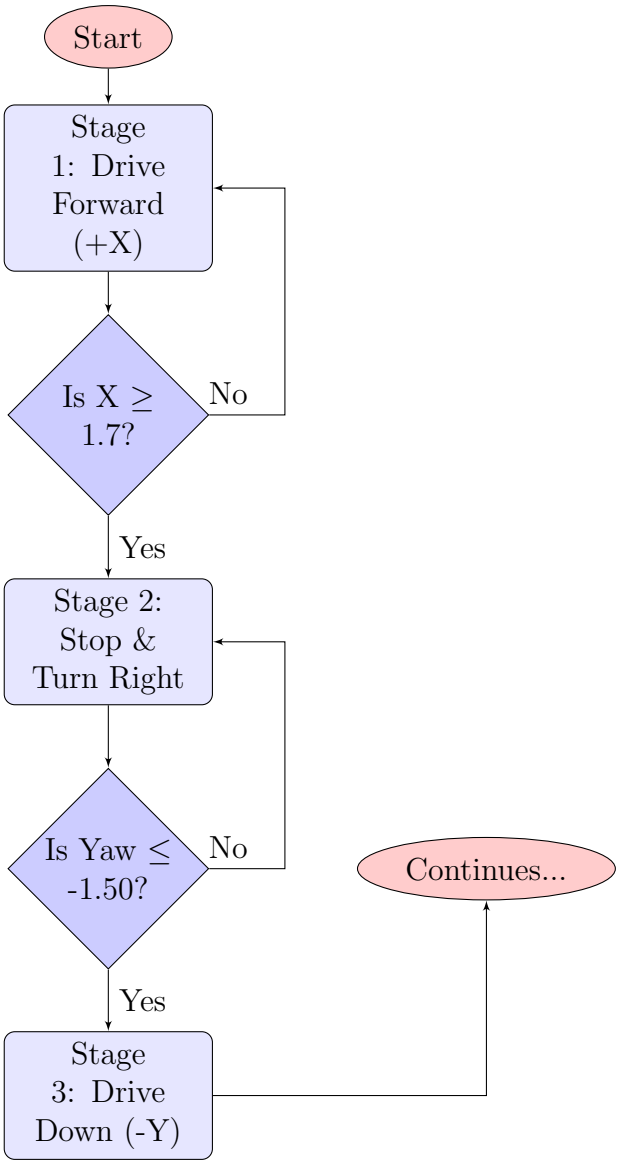

Figure 1: Flowchart of Lab 3 Odometry Logic

The logic implemented in the node `ce315_core_node_lab3.cpp` proceeds as follows:

1. **Stage 1 (Forward):** The robot sets linear velocity to 0.4 m/s. It monitors the X-coordinate, triggering a transition when $x \geq 1.7$ m.
2. **Stage 2 (Turn Right):** Linear velocity is zeroed, and angular velocity is set to -0.4 rad/s. The robot rotates until the yaw reaches -1.50 rad ($\approx -90°$).
3. **Stage 3 (Down):** The robot drives along the negative Y-axis until passing the pillar gap at $y \leq -1.5$ m.
4. **Stage 4 to 6:** The robot executes a diagonal turn and approaches the final marker, stopping when the Euclidean distance is < 0.15 m.

2.2 Trajectory and Velocity Analysis

The robot's movement was recorded and plotted to analyze the effectiveness of the open-loop control strategy.

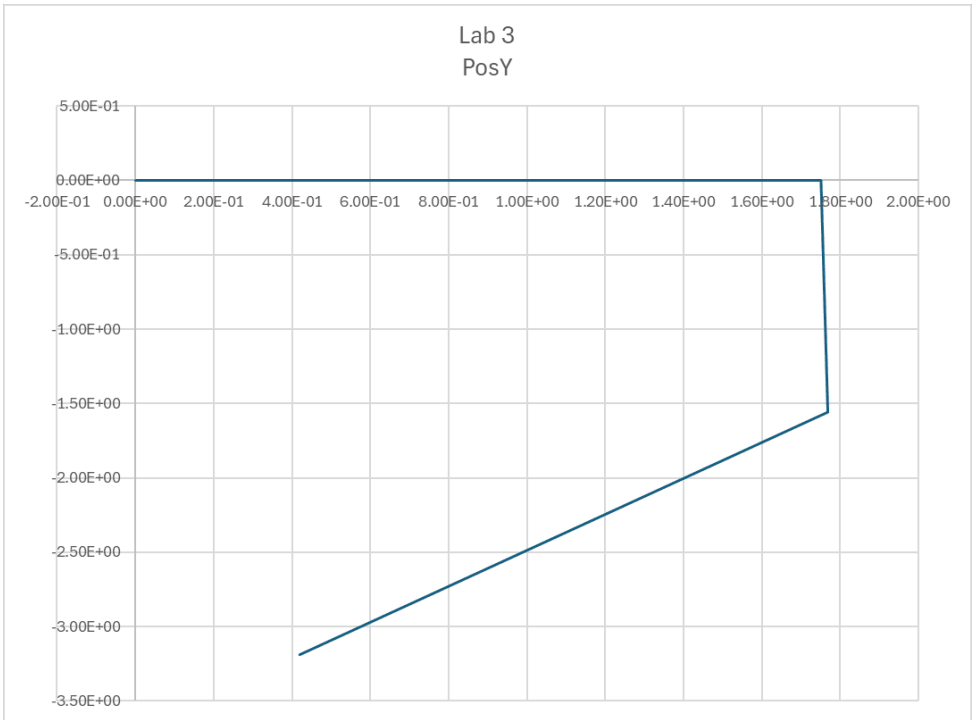

Figure 2: Robot Trajectory (X-Y) using Lab 3 Odometry Control

As shown in Figure 2, the trajectory consists of straight lines connected by sharp, rigid angles.

- **Shape Analysis:** The "Turn-then-Move" logic produces a "rectilinear" trajectory. By isolating translation from rotation, the robot halts entirely prior to pivoting, resulting in a precise 90-degree turn at $x \approx 1.8$.
- **Landmark Selection:** The threshold $x = 1.7$ was selected to clear Pillar T2. The graph shows the actual turn occurs slightly later ($x \approx 1.8$) due to the update cycle latency and robot inertia.

3 Implementation of Laser Mapping in Lab 4

Lab 4 aimed to generate a 2D environmental map by processing LiDAR sensor data. This process required converting raw polar measurements (r, θ) from the scanner into global Cartesian coordinates (x_{map}, y_{map}).

3.1 Mapping Algorithm Flowchart

The mapping process runs inside the `laserScanCallback`. It iterates through the laser range array, checks for valid data, and applies a trigonometric transformation based on the robot's current pose.

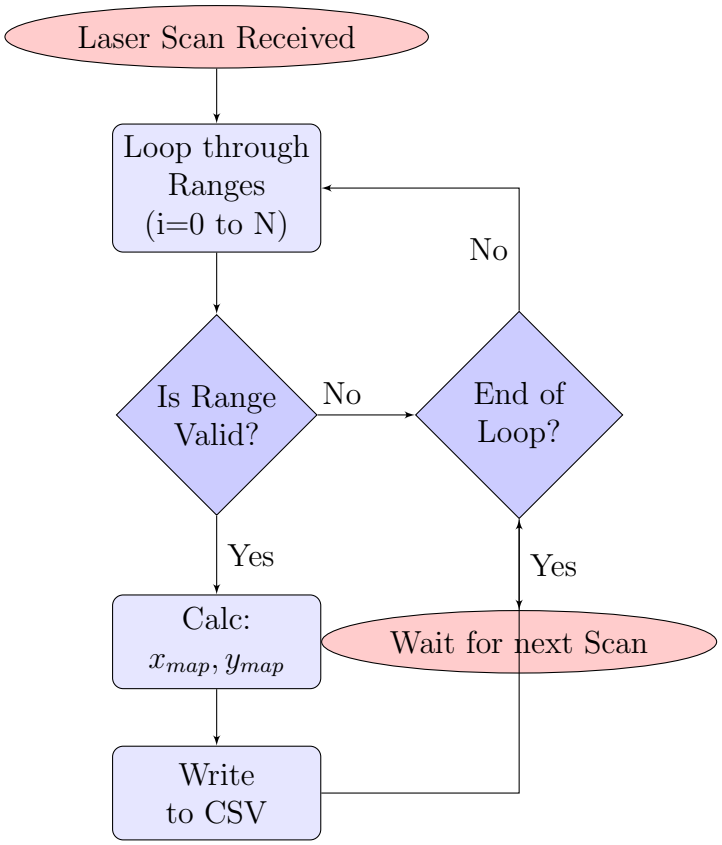

Figure 3: Flowchart of Lab 4 Laser Mapping Algorithm

The transformation equation used in `ce315_core_node_lab4.cpp` is:

$$x_{map} = x_{robot} + r \cdot \cos(\psi + \theta_{scan}) \quad , \quad y_{map} = y_{robot} + r \cdot \sin(\psi + \theta_{scan}) \tag{1}$$

3.2 Analysis of Mapping Data and System Instability

During the execution of Lab 4, a complete graphical representation of the environment map could not be generated due to system instability.

However, the robot successfully recorded its **trajectory data** (velocity profile and position) during the attempt. This data is visualized below:

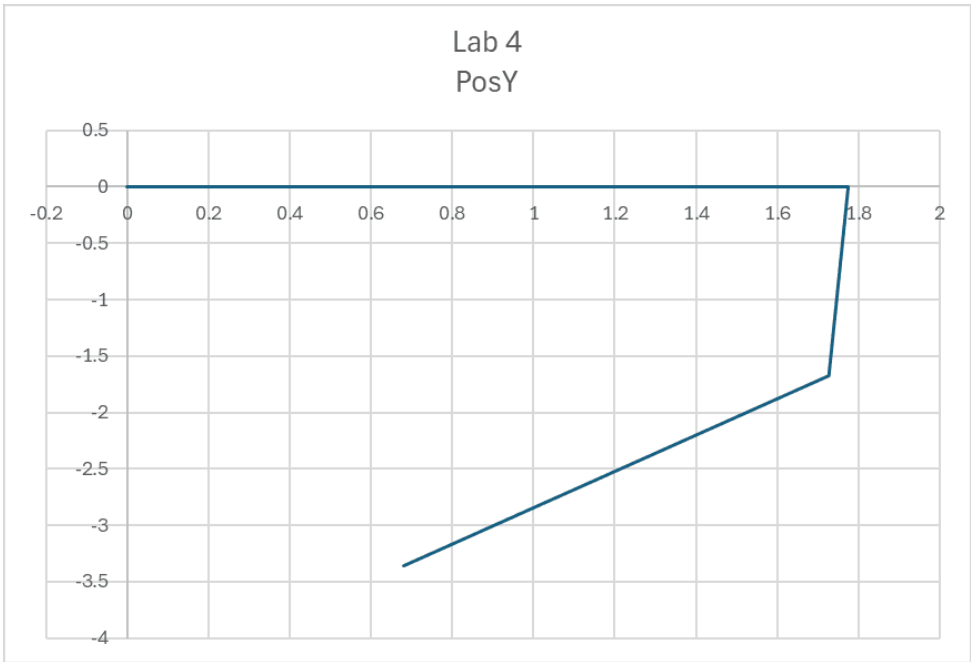

Figure 4: Robot Trajectory during Laser Mapping (Lab 4)

The Map Graph is omitted because the mapping algorithm logged all unfiltered laser data directly to a CSV file. The LiDAR's high frequency and resolution generated an excessive data volume, leading to critical I/O latency and memory overflow. Consequently, the application crashed prior to rendering the map, demonstrating the essential need for the optimization strategies (such as Grid Filtering) detailed in Section 6.

4 Implementation of PID Control Strategy in Lab 5

In Lab 5, a Proportional-Integral-Derivative (PID) controller was implemented to refine the robot's rotational and heading adjustments. This closed-loop architecture replaced the "Bang-Bang" control used in Lab 3, enabling continuous motion during navigation.

4.1 Controller Logic

The control loop calculates the error between the *desired heading* and the *current heading* at every time step. The PID output determines the angular velocity (ω):

$$\omega(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{2}$$

4.2 Trajectory Analysis

The implementation of PID control fundamentally altered the robot's movement characteristics.

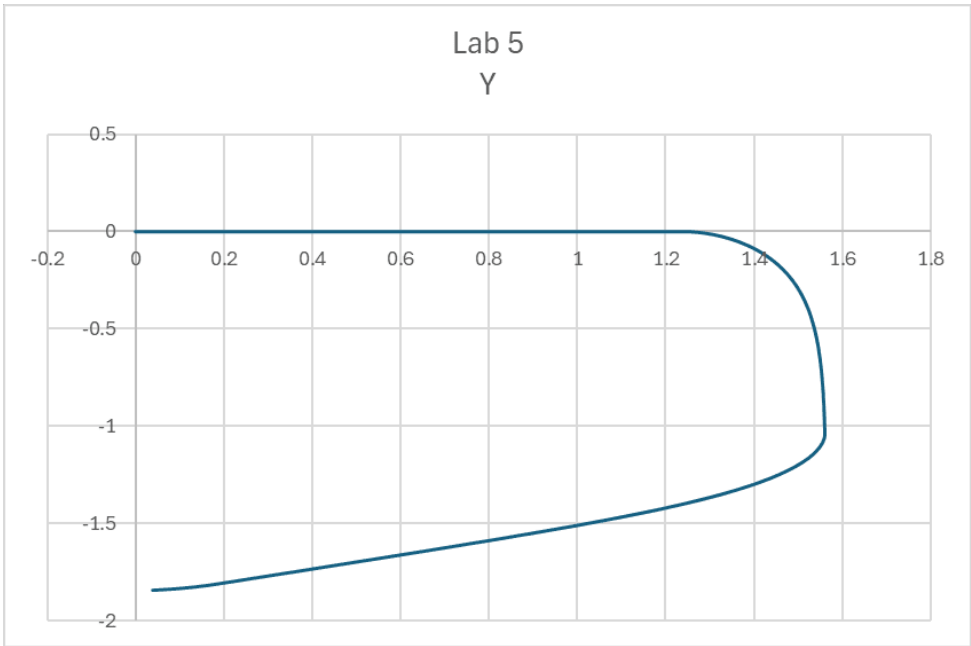

Figure 5: Robot Trajectory with PID Control (Lab 5)

Graph Explanation:

- **Rounded Corners:** Unlike Lab 3, the turn at $X \approx 1.3$ in Figure 5 is a smooth arc. This occurs because the robot maintains a non-zero linear velocity (0.25 m/s) while the PID controller simultaneously adjusts the angular velocity.
- **Overshoot and Correction:** The curve shows a slight overshoot as it transitions into the diagonal segment ($y \approx -1.5$). This is characteristic of the Proportional gain driving the turn, while the Derivative term works to dampen the oscillation.

4.3 Controller Tuning

The gains were selected as follows:

- $K_p = 1.0$**:** This proportional gain provides the primary turning force. A value of 1.0 ensured the robot reacted quickly to heading errors.
- $K_i = 0.0$**:** The integral term was disabled to prevent "wind-up" and instability, which is common in short navigation tasks where error does not accumulate over long periods.
- $K_d = 0.1$**:** The derivative term was added to smooth the approach to the target heading, acting as a "damper" to reduce the oscillation caused by the strong proportional term.

5 Map Improvement and Optimizing Code (Labs 6 - 8)

This section details the improvements made to address the mapping crashes encountered in Lab 4 and the optimization of the control code structure.

5.1 Map Improvements: Grid-Based Filtering

To resolve the application crashing issue caused by excessive data logging, the improved code in `ce315_core_node.cpp` implements a **Grid-Based Filtering** approach.

Revised Algorithm:

- A grid resolution of 0.05m is defined.
- Instead of logging every single laser point, the algorithm calculates integer grid coordinates: $grid\_x = \lfloor global\_x / resolution \rfloor$.
- A `std::set` ('visited_cells') tracks which grid cells have already been mapped.
- **Result:** Duplicate points are discarded. This drastically reduces the file size and memory usage, preventing the crash observed in Lab 4 and producing a cleaner, thinner map representation.

5.2 Trajectory Optimization Results

The code optimizations and refined parameter tuning resulted in the trajectory shown below.

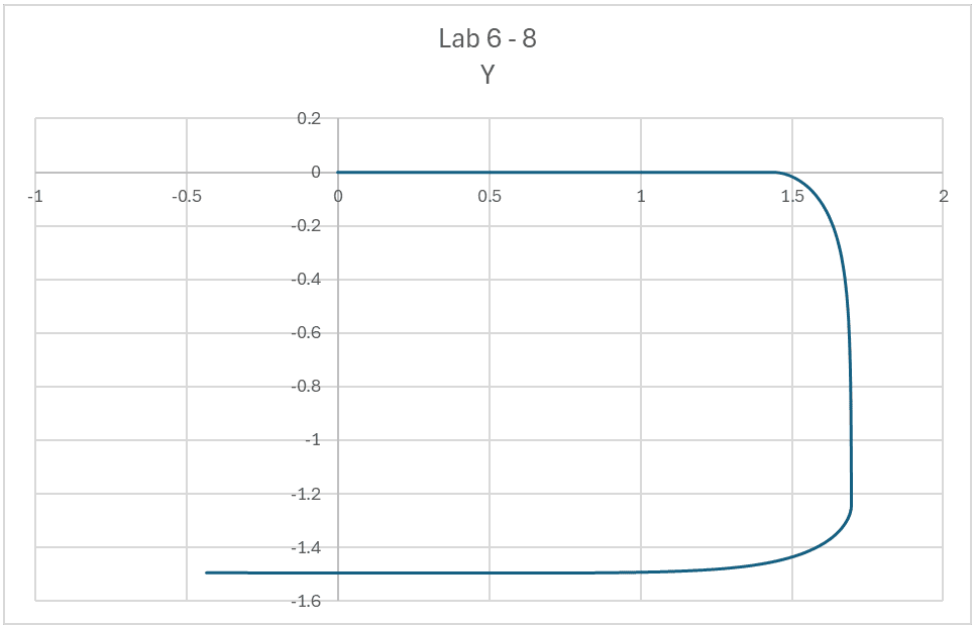

Figure 6: Optimized Robot Trajectory (Lab 6-8)

A comparison of Figure 6 and Figure 5 reveals a more refined turn initiation in the optimized trajectory. By increasing the turn threshold from 1.2 to $x \geq 1.4$, the robot gains additional clearance prior to beginning the arc. This smooth curvature confirms that the optimized PID logic facilitates efficient continuous motion, eliminating the discontinuous "stop-start" behavior observed in previous labs.

5.3 Optimized Code: ce315_core_node.cpp

Below is the fully functional, optimized code. Optimizations are highlighted in yellow.

Listing 1: Optimized Core Node

```
#include <chrono>
#include <memory>
#include <cmath>
#include <limits>
#include <fstream>
#include <iostream>
#include <set>          // OPTIMIZATION: Included for map filtering@
#include <utility>     // OPTIMIZATION: For std::pair@
#include "rclcpp/rclcpp.hpp"
#include "nav_msgs/msg/odometry.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"
#include "geometry_msgs/msg/twist.hpp"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

using namespace std::chrono_literals;

class OptimizedNode : public rclcpp::Node {
public:
    OptimizedNode() : Node("ce315_optimized_node"),
                      stage(1), x(0.0), y(0.0), yaw(0.0),
                      kp(1.0), ki(0.0), kd(0.1),
                      e_prev(0.0), e_int(0.0)
    {
        odom_sub = this->create_subscription<nav_msgs::msg::Odometry>(
            "/odom", 10, std::bind(&OptimizedNode::odomCallback, this,
               std::placeholders::_1));

        laser_sub =
           this->create_subscription<sensor_msgs::msg::LaserScan>(
            "/scan", 10, std::bind(&OptimizedNode::laserScanCallback,
               this, std::placeholders::_1));

        cmd_pub =
           this->create_publisher<geometry_msgs::msg::Twist>("/cmd_vel",
           10);

        timer = this->create_wall_timer(
            100ms, std::bind(&OptimizedNode::controlLoop, this));

        vel_log.open("optimized_velocity.csv");
        vel_log << "Time,Linear,Angular,X,Y,Yaw,Stage\n";

        map_log.open("optimized_map.csv");
        map_log << "MapX,MapY\n";

         grid_resolution = 0.05; // OPTIMIZATION: Defined grid
           resolution@
    }

    ~OptimizedNode() {
        if (vel_log.is_open()) vel_log.close();
        if (map_log.is_open()) map_log.close();
    }

```

```
private:
    void odomCallback(const nav_msgs::msg::Odometry::SharedPtr msg) {
        x = msg->pose.pose.position.x;
        y = msg->pose.pose.position.y;

        double qx = msg->pose.pose.orientation.x;
        double qy = msg->pose.pose.orientation.y;
        double qz = msg->pose.pose.orientation.z;
        double qw = msg->pose.pose.orientation.w;
        double siny_cosp = 2.0 * (qw * qz + qx * qy);
        double cosy_cosp = 1.0 - 2.0 * (qy * qy + qz * qz);
        yaw = std::atan2(siny_cosp, cosy_cosp);
    }

    // OPTIMIZATION: Improved Mapping with Grid Filter
    void laserScanCallback(const sensor_msgs::msg::LaserScan::SharedPtr
        msg) {
        if (!map_log.is_open()) return;

        for (size_t i = 0; i < msg->ranges.size(); ++i) {
            float r = msg->ranges[i];

            if (std::isinf(r) || std::isnan(r) || r < msg->range_min ||
                r > msg->range_max)
                continue;

            double angle = msg->angle_min + (i * msg->angle_increment);
            double global_x = x + (r * std::cos(yaw + angle));
            double global_y = y + (r * std::sin(yaw + angle));

            // OPTIMIZATION: Discretize coordinates to grid@
             int grid_x = std::floor(global_x / grid_resolution);
             int grid_y = std::floor(global_y / grid_resolution);
             std::pair<int, int> grid_cell = {grid_x, grid_y};

             // OPTIMIZATION: Only log if cell not visited@
            if (visited_cells.find(grid_cell) == visited_cells.end()) {
                map_log << global_x << "," << global_y << "\n";
                visited_cells.insert(grid_cell);
            }
        }
    }

    // OPTIMIZATION: Helper function to calculate PID@
     double calculatePID(double target_yaw) {
        double error = target_yaw - yaw;
        while (error > M_PI) error -= 2.0 * M_PI;
        while (error < -M_PI) error += 2.0 * M_PI;

        e_int += error;
        double e_der = error - e_prev;
        e_prev = error;

        return (kp * error) + (ki * e_int) + (kd * e_der);
    }

    void controlLoop() {
        geometry_msgs::msg::Twist cmd;
```

```
108         double target_yaw = 0.0;
109         double target_speed = 0.0;
110 
111          // OPTIMIZATION: Switch statement for cleaner FSM@
112         switch (stage) {
113             case 1: // Move Straight
114                 target_yaw = 0.0;
115                 target_speed = 0.2;
116                 if (x >= 1.4) stage = 2;
117                 break;
118 
119             case 2: // Turn Right (Drive Down)
120                 target_yaw = -1.57; // -90 deg
121                 target_speed = 0.2;
122                 if (y <= -1.2) stage = 3;
123                 break;
124 
125             case 3: // Approach (Straight Left/West)
126                 target_yaw = 3.14;
127                 target_speed = 0.2;
128                 if (x <= -0.85) stage = 4;
129                 break;
130 
131             case 4: // Final Alignment to Red Dot
132                 target_yaw = 3.14;
133                 target_speed = 0.15;
134                 if (x <= -2.1) stage = 5;
135                 break;
136 
137             case 5: // Stop
138                 target_speed = 0.0;
139                 target_yaw = yaw;
140                 break;
141 
142             default:
143                 target_speed = 0.0;
144                 break;
145         }
146 
147         if (stage != 5) {
148             cmd.angular.z = calculatePID(target_yaw); // Use helper@
149             cmd.linear.x = target_speed;
150         } else {
151             cmd.angular.z = 0.0;
152             cmd.linear.x = 0.0;
153         }
154 
155         cmd_pub->publish(cmd);
156 
157         if (vel_log.is_open()) {
158             vel_log << this->now().seconds() << "," << cmd.linear.x <<
                    ","
159                     << cmd.angular.z << "," << x << "," << y << "," <<
                        yaw << "," << stage << "\n";
160         }
161     }
162 
163     // Class Variables
```

```
164     rclcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odom_sub;
165     rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr
            laser_sub;
166     rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr cmd_pub;
167     rclcpp::TimerBase::SharedPtr timer;
168
169     std::ofstream vel_log, map_log;
170
171     int stage;
172     double x, y, yaw;
173     double kp, ki, kd;
174     double e_prev, e_int;
175
176     double grid_resolution;
177     std::set<std::pair<int, int>> visited_cells;
178 };
179
180 int main(int argc, char **argv) {
181     rclcpp::init(argc, argv);
182     rclcpp::spin(std::make_shared<OptimizedNode>());
183     rclcpp::shutdown();
184     return 0;
185 }
```

6 Conclusion

This report successfully demonstrated the implementation of a mobile robot navigation system developed within the ROS and Gazebo frameworks. The pedagogical progression from fundamental odometry-based control, as seen in Lab 3, to a refined Proportional-Integral-Derivative (PID)-based navigation system in Lab 5 underscored the critical role of feedback control loops in achieving enhanced motion accuracy and smoothness. Furthermore, the evolution of the mapping algorithm illustrated effective strategies for sensor data optimization. The transition from processing raw, unfiltered point clouds in Lab 4—which ultimately led to data volume overload and system instability—to the final implementation's filtered, grid-based methodology highlighted practical solutions to common challenges in environmental perception. The resulting optimized navigation node seamlessly integrates these components, enabling the robot to reliably traverse the predefined path. Potential future enhancements to the system could involve the integration of Simultaneous Localization and Mapping (SLAM) techniques to comprehensively mitigate the odometric drift observed during experimental trials.